Object-Oriented Programming I

Lecture 3: Encapsulation

Dr. Ghemougui Abdessettar

University of M'Sila – Faculty of Mathematics and Computer Science

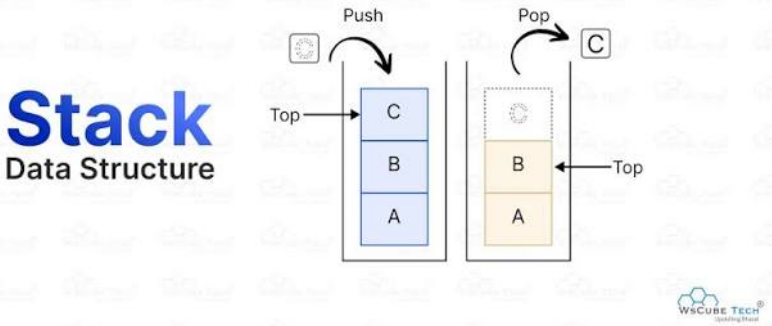
October 13, 2026

Lecture plan

- Defining the Stack data structure in procedural programming
- Limitations of the procedural approach
- Implementing the Stack as an object
- Encapsulation

Stack Data Structure

A stack stores elements using **LIFO** (*Last In, First Out*): the last element pushed is the first one removed.



Let us implement a stack in C using an array and an index.

Stack in Procedural Programming

We group related data in a `struct` and write separate functions that operate on that data.

```c
#include <stdio.h>
#define CAPACITY 100

typedef struct {
  int mem[CAPACITY];
  int top; /* number of elements; mem[top - 1] is the top element */
} Stack;
```

Stack in Procedural Programming

Operation	Description
`init`	initialize the stack
`is_empty`	is the stack empty?
`is_full`	is the stack full?
`push`	push an element onto the stack
`pop`	pop an element from the stack
`print_stack`	print the contents of the stack

Operations: Prototypes

```
1 void init(Stack *s);
2 int  is_empty(const Stack *s);
3 int  is_full(const Stack *s);
4 void push(Stack *s, int value);
5 int  pop(Stack *s);
6 void print_stack(const Stack *s);
```

Initialization, Empty, and Full

First, we initialize the stack:

```
1 void init(Stack *s) {
2   s->top = 0; // when top == 0 the stack is empty
3   for (int i = 0; i < CAPACITY; i++)
4     s->mem[i] = 0;
5 }
```

Then we check whether the stack is empty or full:

```
1 int is_empty(const Stack *s) { return s->top == 0; }
2 int is_full(const Stack *s)  { return s->top == CAPACITY; }
```

Push

To push an element, we first check that the stack is not full:

```
1 void push(Stack *s, int value) {
2   if (is_full(s)) {
3     fprintf(stderr, "error: stack is full\n");
4     return;
5   }
6   s->mem[s->top] = value; // put the value in the first free slot
7   s->top = s->top + 1;    // increment top
8 }
```

Pop

To pop an element, we first check that the stack is not empty:

```
1 int pop(Stack *s) {
2   if (is_empty(s)) {
3     fprintf(stderr, "error: stack is empty\n");
4     return 0; // ambiguous: 0 is also a legitimate value
5   }
6   s->top = s->top - 1;
7   return s->mem[s->top];
8 }
```

Using (and Misusing) the Stack

```c
int main(void) {
  Stack s;          /* declared but not initialized yet */
  init(&s);         /* an operation on the data, called separately */
  print_stack(&s); /* data and operations are separate: we pass &s every
    time */
  push(&s, 5);
  push(&s, 4);
  print_stack(&s);                /* 5 4 */
}
```

Using (and Misusing) the Stack

```c
int main(void) {
  Stack s;          /* declared but not initialized yet */
  init(&s);         /* an operation on the data, called separately */
  push(&s, 5);
  pop(&s);
  pop(&s);          /* pop from an empty stack: error message, returns 0 */
  s.top = 10;       /* uncontrolled state modification */
  s.mem[4] = 23;    /* uncontrolled state modification */
  print_stack(&s); /* prints 5 0 0 0 23 0 0 0 0 0 */
  return 0;
}
```

Problems in the Procedural Implementation

- **Data and operations are separated**
  The stack and the operations that work on it are independent, so we must pass the stack as an argument on every call.

Problems in the Procedural Implementation

- **Data and operations are separated**
  The stack and the operations that work on it are independent, so we must pass the stack as an argument on every call.
- **No data protection**
  The data can be changed in an uncontrolled way: a programmer can directly modify the `top` index or the stored values.

Problems in the Procedural Implementation

- **Data and operations are separated**
  The stack and the operations that work on it are independent, so we must pass the stack as an argument on every call.
- **No data protection**
  The data can be changed in an uncontrolled way: a programmer can directly modify the `top` index or the stored values.
- **No initialization guarantee**
  The data can be used in an uninitialized state, for example if the programmer forgets to call `init()`.

Stack as a Class

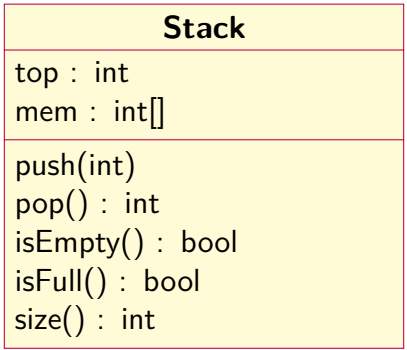
A **class** groups data together with the operations that work on that data.

- Data members are called **attributes**
- Functions that belong to a class are called **member functions** or **methods**

A stack can be seen as an object that has a **state** and a **behavior**:

- **State**: elements, size, capacity
- **Behavior**: `push`, `pop`, `isEmpty`, `isFull`

UML Class Diagram



Defining a Class in C++

We create a class with the keyword `class`. Attributes are declared inside the class body, like fields in a C `struct`.

```
#define CAPACITY 100

class Stack {
public:
  // attributes
  int mem[CAPACITY]{};
  int top = 0;
};
```

Adding a Method

Methods are declared inside the class body as member functions.

```
1 class Stack {
2 public:
3   int mem[CAPACITY]{};
4   int top = 0;
5
6   bool isEmpty() {
7     return top == 0; // the stack is empty when top is 0
8   }
9 };
```

Adding a Method

```
1 class Stack {
2 public:
3   int mem[CAPACITY]{};
4   int top = 0;
5   bool isEmpty() {
6     return top == 0; // the stack is empty when top is 0
7   }
8 };
```

Note

`isEmpty` accesses the attribute `top` without receiving any argument: it operates on the object it was called on.

The Complete Class

```
1  class Stack {
2  public:
3    // attributes
4    int mem[CAPACITY]{};
5    int top = 0;
6
7    // methods
8    bool isEmpty() { return top == 0; }
9    bool isFull()  { return top == CAPACITY; }
10   int  size()    { return top; }
11   // ...
```

The Complete Class

```
1
2  // ...
3    void push(int value) {
4      if (isFull()) return;
5      mem[top++] = value;
6    }
7
8    int pop() {
9      if (isEmpty()) return 0;
10     return mem[--top];
11   }
12 };
```

Using the Class Stack

A class defines a new **data type**. Usually we do not use the class itself but an **instance** (an **object**) of the class.

```
#include <iostream>

int main() {
  Stack s1;     // s1 is an instance (object) of the class Stack
  Stack s2;     // s2 is another instance of the class Stack
  s1.push(10); // call a method on the object
  s2.push(5);
  s1.push(8);
  std::cout << "Stack size: " << s1.size() << std::endl; // 2
}
```

- `s1.push(10)` calls the method `push` of the class `Stack` on the instance `s1`.

Using the Class Stack

Note

`std::cout` is the C++ way of printing to standard output. It plays the role of `printf`.

Data Protection: Still a Problem

We solved the separation of data and operations by grouping them in a class. **What about data protection?**

```
int main() {
  Stack s1, s2;
  s1.push(10);
  s1.top = -1; // direct access to the state
  std::cout << "Stack size: " << s1.size() << std::endl;
}
```

Data Protection: Still a Problem

We solved the separation of data and operations by grouping them in a class. **What about data protection?**

```
1 int main() {
2   Stack s1, s2;
3   s1.push(10);
4   s1.top = -1; // direct access to the state
5   std::cout << "Stack size: " << s1.size() << std::endl;
6 }
```

Warning

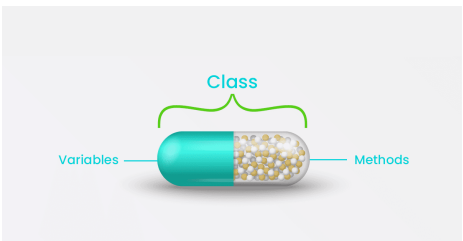
This prints a stack size of −1: an obviously invalid **state** for a stack.

Encapsulation

To avoid this problem, we introduce a key concept of OOP: **encapsulation**.

Definition

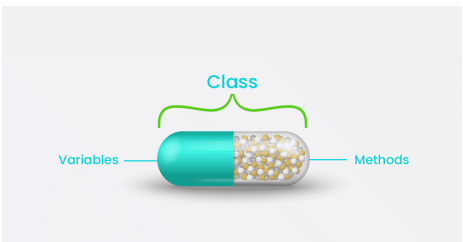
Encapsulation is the process of bundling an object's data and the methods that operate on that data into a single unit (a class), while restricting direct access to the internal state and implementation.



Encapsulation

To avoid this problem, we introduce a key concept of OOP: **encapsulation**.

Encapsulation acts as a protective barrier around data and logic. It hides internal implementation details and prevents outside code from modifying the state in unexpected ways.



Access Modifiers: Public and Private

Many programming languages provide access modifiers such as `public` and `private` (called *access specifiers* in C++). In C++:

- Members (attributes or methods) declared after `public:` are accessible from outside the class
- Members declared after `private:` are accessible only from within the class
- Accessing a private member from outside the class causes a **compilation error**

Note

In a C++ `class`, members are private by default. In a `struct`, they are public by default.

Access Modifiers: Public and Private

- Members (attributes or methods) declared after `public:` are accessible from outside the class
- Members declared after `private:` are accessible only from within the class
- Accessing a private member from outside the class causes a **compilation error**

In our example, the state of the stack is represented by `mem` and `top`: we must prevent any external access to them.

Making the State Private

```
1  class Stack {
2  private:
3    // attributes = state
4    int mem[CAPACITY]{};
5    int top = 0;
6  public:
7    // methods = behavior
8    bool isEmpty() { return top == 0; }
9    bool isFull()  { return top == CAPACITY; }
10   int  size()    { return top; }
11   void push(int value) { if (isFull()) return; mem[top++] = value; }
12   int  pop() { if (isEmpty()) return 0; return mem[--top]; }
13 };
```

The Compiler Now Protects the State

```
1 int main() {
2   Stack s1, s2;
3   s1.push(10);
4   s2.push(5);
5   s1.push(8);
6   s1.top = -1; // compilation error
7   std::cout << "Stack size: " << s1.size() << std::endl;
8 }
```

The Compiler Now Protects the State

```
1 s1.top = -1; // error: 'top' is a private member of 'Stack'
```

The internal state of the stack can now only be updated by the class itself, through its own methods. An object's state should only be changed through its behavior. Every change of state must be justified by one of the object's methods.

So What Is Encapsulation?

So What Is Encapsulation?

- Bundling data and operations together

So What Is Encapsulation?

- Bundling data and operations together
- Hiding the internal state

So What Is Encapsulation?

- Bundling data and operations together
- Hiding the internal state
- Controlling access to the internal state through public methods

So What Is Encapsulation?

- Bundling data and operations together
- Hiding the internal state
- Controlling access to the internal state through public methods
- Protecting the object's **invariants**

So What Is Encapsulation?

- Bundling data and operations together
- Hiding the internal state
- Controlling access to the internal state through public methods
- Protecting the object's **invariants**

Invariant

A condition that must always hold for a valid object. For the stack:
$0 \leq \texttt{top} \leq \texttt{CAPACITY}$.

Encapsulation in the Stack Code

Idea	**Where it appears in the code**
Bundle data + behavior	`mem[]` and `top` live *inside* `Stack`, together with `push`/`pop`/`size`.
Hide internal state	`mem` and `top` are `private`: `main()` cannot write `s1.top = -5;` or `s1.mem[0] = 99;`.
Controlled access via public methods	The only way to change the stack is `push()` / `pop()`.
Invariants protected	Outside code cannot touch `top`, so it can never be negative or greater than `CAPACITY`. `pop()` on empty and `push()` on full are handled *inside* the class.

Course Materials



`https://aguemoug.github.io/oop-course/`

Lecture notes, slides, labs, and source code